

Sorting Algorithms & Fundamental Data Structures

Sorting Algorithms

Sorting happens in the program in the following method of the `GeneralManager` class.

```
public void refreshUI() {
    Map<UIDataName, Object> searchData = getSearchData();
    System.out.println("Search Data: " + searchData);
    Table filteredTable = searchEngine.filterItems(searchData);
    Table sortedTable = filteredTable;
    try {
        sortedTable = Sorter.sortTable(filteredTable, (RowComparator)
searchData.get(UIDataName.SORTING_OPTION));
    } catch (ClassCastException e) {
        System.out.println("Sorting option provided: " +
searchData.get(UIDataName.SORTING_OPTION) + " is not a valid RowComparator.
Skipping sorting.");
    }
    List<String> sortedAndFilteredItemIds =
sortedTable.stringColumn(TableColumnName.ID.getColumn()).asList();
    uiManager.updateSearchedItemPagination(sortedAndFilteredItemIds);
}
```

The sorting is done in the `Sorter` class, which uses `InsertionSort` for lists of size less than or equal to 25, and `QuickSort` for lists of size greater than 25. This is because `InsertionSort` is more efficient for small lists, while `QuickSort` is more efficient for larger lists. As stated in other documentation, 25 is arbitrary. In all cases for data in the project except a single size list, `InsertionSort` is faster, but the concept of using different sorting algorithms for different sizes of data is still demonstrated by using a cutoff for picking algorithms.

- Choice of Algorithms
 - `InsertionSort` was chosen for small lists because it has a low overhead and is efficient for small data sets. It has a time complexity of $O(n^2)$ in the worst case, but for small lists, this is not a significant issue.
 - `QuickSort` was chosen for larger lists because it has an average time complexity of $O(n \log n)$ and is generally faster than other sorting algorithms for larger data sets. It also has good cache performance, which can further improve its efficiency.
 - The reason that `QuickSort` is not used for all sorting is because of the overhead associated with the recursive calls
 - An analysis was done on all the sorting algorithms implemented in the project, and it was found that `InsertionSort` is faster in nearly all cases. The cutoff of 25 was chosen arbitrarily, but it is based on the idea that `InsertionSort` is more efficient for small lists, while `QuickSort` is more efficient for larger lists. In practice, the cutoff could be adjusted based on testing to find the point at which `QuickSort` becomes more efficient. Analysis can be seen [HERE](#).
 - An alternative approach attempted was to combine both `QuickSort` and `InsertionSort` into a single algorithm, where `QuickSort` would be used for larger partitions of the data, and

InsertionSort would be used for smaller partitions. This was due to research showing this as a popular approach to sorting. While it was not too difficult to implement, it was nearly identical in speed to **QuickSort**, and only faster than **InsertionSort** for a few sizes, all of which were 25 or less.

- A graph linked [HERE](#) shows that **MixedSort** is very close to **QuickSort**, and that **InsertionSort** is much faster than either other algorithm.

Fundamental Data Structures

• Queues

- A queue is used in the implementation of Dijkstra's algorithm in the **SimilarItemsGraph** class. The queue is used to keep track of the next item to visit in the graph. It is a priority queue, which allows for efficient retrieval of the next item with the lowest distance.
- A queue is also used in the **RecentlyViewedQueue** class. A queue is used here because we want to track the order of recently viewed items and remove the oldest item when the queue reaches its capacity.
 - Implementation Details
 - It is implemented with a **ArrayBlockingQueue** due to the built-in functionality for a fixed size queue and the array backing. This allows for efficient addition and removal of items from the queue in addition to ensuring a fixed maximum size. When data is added to the queue and the queue is at capacity, the oldest item in the queue is automatically removed to make room for the new item. When an item is added that exists already, it is removed and placed back on top.

• LinkedList

- A linked list is used in file **SearchedItemsLinkedList** to handle different pages of items in the search results. This allows for easy navigation between pages, as well as easy insertion of new pages when the search results change.
- Implementation Details
 - **Item** entities are stored instead of **SearchedItemPanel** UI nodes to save on RAM. The UI nodes are generated on the fly when a page is displayed based on the **Item** data in that page's node.
 - Originally, a circular list with preloading for pages within a radius of the current page was implemented, but this was removed to save on RAM, as it was found that the rendering 20+ more **SearchedItemPanel** UI nodes killed the RAM.
 - To allow for a **next** and **previous** page function, the linked list is doubly linked, as this allows for easy traversal in both directions.

• Arrays

- The best use of arrays is within the **StarRatingFilter** class. Here, an array is used to make hold the star labels and make it easy to assign functionality to each star. The array allows for each star to be referenced by its index, which is also used to set the character to either a filled or empty star easily.
- Arrays are also used in the **PriceTree** class, as it returns an array of indices upon computing a submap. This array is used to get the item indexes of price-filtered items. This is useful because when filtering the table by the accepted indices, the following is used.

- ```
filteredTable.where(Selection.with(itemIndicesInRange));
```
- In this code snippet, `itemIndicesInRange` is the array returned by the call to `PriceTree`, which passes through `PriceFilter`. This allows for filtering the table directly from the result of the call to `PriceTree` and `PriceFilter`.
- Flow
  - `SearchEngine` calls `PriceFilter` to filter the table by price.
  - `PriceFilter` calls `PriceTree` to get the item indices of items within the price range.
  - `PriceTree` uses the `subMap` method of the `TreeMap` to get a view of the portion of the tree that falls within the price range, and then traverses this portion of the tree to collect the nodes whose prices are in the range. The nodes are then mapped to their values (item indices), which are then collected into an array and returned to `PriceFilter`.
  - `PriceFilter` then returns this array of item indices to `SearchEngine`, which uses it to filter the table of items by price.